НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): Студент групи АІ-241
Албул Віра Петрівна

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Киев	Київ
eval_count	316	5
Час, с	14,50	0,12
Токенів/с	21,79	42,07

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1183,4	166,6
Час, с	47,61	5,64

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Токени пішли на внутрішню самоперевірку та генерацію «ланцюжка думок» (Chain-of-Thought), який записується в поле thinking. Модель декілька разів перевіряла гіпотези щодо столиці (переконувалася, що це не Львів, Харків чи Криж), перш ніж сформувати підсумкову відповідь у полі response.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі дають вирішальну перевагу в багатокрокових математичних задачах, формальній логіці, аналізі складного коду та системному плануванні. Їх використання повністю невиправдане на простих фактичних запитах

(пошук фактів, переклад, класифікація), оскільки вони дають таку ж якість, але витрачають у 10–50 разів більше часу й токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивна модель навчається переважно методом тонкого налаштування з вчителем (SFT) на парах «інструкція — пряма відповідь». Міркувальна модель додатково навчається за допомогою навчання з підкріпленням (Reinforcement Learning), де її винагороджують за побудову деталізованого, логічно послідовного та самокритичного ланцюжка роздумів перед видачею фінальної відповіді.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, це означає лише те, що модель виділяє собі більше обчислювальної «робочої пам'яті» під час інференсу. У складних задачах (як наша задача на логіку, де qwen3:4b показала 100% точності проти 20% у llama3.2:3b) це дозволяє уникнути інтуїтивних помилок, але на простих задачах це просто надлишкові обчислення.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: llama3.2:3b видає близько 42 токенів/с, а qwen3:4b — близько 22–26 токенів/с. Це пояснюється різницею у розмірі моделей (3 млрд параметрів проти 4 млрд), а також внутрішньою архітектурою та механізмами уваги (Attention), які у qwen3:4b потребують більших обчислювальних витрат на кожен токен.

Висновок за завдання 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, але витрати токенів відрізняються у 30,6 разів. На задачі про комбінаторику та розрахунки ситуація протилежна – саме розписаний хід думок дозволив уникнути помилки. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 300,8 с проти 39,0 с, що робить міркувальну модель непридатною для швидкого налагодження. Інструктивні моделі

орієнтовані на миттєве виконання команд і генерують пряму відповідь, що робить їх ідеальними, швидкими та дешевими для фактів, форматування й діалогів. Міркувальні моделі формують внутрішній ланцюжок думок. Це вимагає в десятки разів більше часу й токенів, але є незамінним для складних логічних, математичних та багатокрокових задач. Використовувати міркувальну модель для простих запитів невиправдано через марну перевитрату ресурсів.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Проведено дослідження впливу системного інструктування (system-повідомлення) на обмеження довжини, мови відповіді та витрату токенів для моделей llama3.2:3b та qwen3:4b.

Таблиця 3 – Вплив system-повідомлення

Модель	**System**	**Довжина відповіді, символів**	**Мова відповіді**	**Кількість речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	1174	українська (з суржиком та калькою)	~10 + блоки коду	Ні
llama3.2:3b	українською, 2 речення	270	українська	2	Так
qwen3:4b	лаконічний асистент	267	українська	4 пункти	Так
qwen3:4b	українською, 2 речення	~160	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	**System**	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	54	463	517
llama3.2:3b	українською, 2 речення	58	77	135
qwen3:4b	лаконічний асистент	30	260	290
qwen3:4b	українською, 2 речення	34	210	244

Аналітичні питання по експерименту:

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
Відповідь: Обидві моделі при прямому системному промпті змогли сформувати відповідь рівно з двох речень, проте llama3.2:3b скоротила обсяг з 1174 до 270 символів, чітко згенерувавши рівно 2 речення.
2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
Відповідь: Прямих переходів на англійську мову не було, проте у llama3.2:3b спостерігалася значна кількість кальок з російської мови та вигаданих слів (наприклад, «зумуються», «якомисією», «початковийндекс», «демонustryють»), що свідчить про слабку якість українського датасету в цій моделі.
3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
Відповідь: prompt_tokens зросли (з 54 до 58 токенів у Llama та з 30 до 34 у Qwen). Це пояснюється тим, що друге системне повідомлення виявилося довшим за кількістю символів і токенів.
4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?
Відповідь: В qwen3:4b загальна кількість completion_tokens включає не лише видимий текст у полі content, але й прихований внутрішній блок міркувань thinking (Chain-of-Thought), який модель генерує перед видачею фінального тексту.
Питання до чат-бота:
1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?
Відповідь: Різні моделі мають різний рівень здатності слідувати інструкціям (instruction-following), що залежить від їхньої архітектури, розміру та якості тонкого налаштування. Міркувальні моделі перед формуванням виводу детально аналізують власні кроки та перевіряють виконання умов, тому точніше витримують задані рамки. Базові інструктивні моделі просто генерують найбільш

ймовірні наступні токени, через що швидше гублять жорсткі обмеження за довжиною чи мовою.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення визначає глобальну роль, базові правила поведінки та межі дозволеного на увесь час діалогу, тоді як user-повідомлення задає лише конкретне поточне завдання. Більшість сучасних моделей під час навчання з підкріпленням спеціально налаштовують надавати найвищий пріоритет саме системним інструкціям для дотримання безпеки та заданого формату.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Це зумовлено формуванням внутрішнього «ланцюжка думок» (Chain-of-Thought), який міркувальна модель генерує перед виведенням підсумкового тексту. Навіть якщо користувач отримує коротку відповідь із двох речень, модель витрачає сотні вихідних токенів на приховану чернетку, де аналізує задачу, планує хід розв'язання та перевіряє результати.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Добре натренована та якісно відкалібрована модель надасть перевагу інструкції із system-повідомлення, оскільки воно має вищий системний пріоритет. Однак малі або слабше оптимізовані моделі в ситуації конфлікту ролей можуть заплутатися, згенерувати галюцинації або піддатися впливу наполегливого запиту від користувача.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі (на 3–4 млрд параметрів) мають обмежений словниковий запас і під час тренування бачили суттєво меншу частку текстів українською мовою. Крім того, стандартні токенізатори розбивають слова менш поширених

мов на дрібніші фрагменти, через що модель втрачає цілісний контекст і починає вставляти кальки, русизми або навіть символи з інших алфавітів.

Висновок за завдання 2:

Дослідження довело, що жорстке системне повідомлення здатне кардинально змінювати обсяг вихідних токенів, але ефективність виконання обмежень прямо залежить від здатності моделі дотримуватися інструкцій. Для інструктивної моделі детальний системний промпт скоротив обсяг генерації з 463 до 76 токенів, однак це призвело до галюцинацій та вставки сторонніх символів. Для міркувальної моделі системне обмеження вимагало додаткового внутрішнього аналізу, що забезпечило бездоганне дотримання формату. System-повідомлення задає глобальні правила й роль моделі, проте надійність його виконання залежить від архітектури та розміру нейромережі. Малі інструктивні моделі під жорсткими обмеженнями схильні деградувати й галюцинувати, тоді як міркувальні моделі дотримуються системних інструкцій значно точніше. Самої лише інструкції для гарантованого формату часто недостатньо; якщо модель її ігнорує, варто надати приклади, сформулювати прямі заборони, використати програмну валідацію або перейти на потужнішу модель.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Розроблено уніфікований модуль providers.py для роботи з локальною (Ollama) та хмарною (Groq) моделями через єдиний OpenAI-сумісний інтерфейс API.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	**Локальна**	**Хмарна**
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	13,05	0,93
completion_tokens	199	271
Довжина відповіді, символів	582	601
Мова відповіді	Українська (з сильним суржиком та білоруськими/російськими калькованими словами)	Грамотна українська

Фактичних помилок у відповіді	Повні галюцинації (назвала обидві структури «Сторона», переплутала LIFO та FIFO, навела нісенітниці замість методу доступу)	Без помилок (чітко вказано LIFO/FIFO, append/pop та використання collections.deque)

Аналітичні питання по експерименту:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: 0 рядків у самому коді виклику. Вся різниця зводиться до зміни 3 параметрів у словнику конфігурації провайдерів (base_url, api_key, model).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель надала ідеально точну та практичну відповідь. Вона чітко розмежувала LIFO для стека та FIFO для черги, а також навела коректний Python-код з використанням collections.deque.popleft(). Локальна модель згенерувала беззмістовний текст із дубльованими заголовками «Сторона».

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, локальна модель продемонструвала повну деградацію української мови та фактажу: використала неіснуюче слово «змагнення», перекрутила назви термінів та не змогла пояснити алгоритмічну суть даних.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарний сервер використовує високопродуктивне залізо (спеціалізовані тензорні прискорювачі з високою пропускною здатністю пам'яті), де обчислення 120B моделі відбуваються за сотні мілісекунд. Локальний ПК виконував інференс на звичайному процесорі/ОЗП, де швидкість обмежена шиною пам'яті комп'ютера.

Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою випустила комерційно успішний API для чат-моделей. Розробники масово написали під нього бібліотеки та інфраструктуру. Інші сервіси адаптували цей формат, щоб розробники могли перемикати провайдерів зміною лише base_url та api_key без переписання коду.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір моделі є вирішальним фактором для знання мови, складних міркувань та запобігання галюцинаціям. Малі моделі (3B) мають обмежену місткість знань і при складних запитах чи рідкісних мовах (українська) втрачають зв'язність, тоді як великі моделі (120B) володіють багатим мовним та технічним контекстом.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Мережева затримка (RTT) додає лише 50–150 мс до часу до першого токена (TTFT). При цьому швидкість виводу токенів на серверних прискорювачах сягає 200–500 токенів/с, тоді як локальний процесор видає лише 15 токенів/с. На обсязі від 100 токенів хмарна модель суттєво випереджає локальну.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Вхідні дані надсилаються на сторонні сервери, де вони можуть зберігатися в логах або використовуватися для донавчання моделей. У комерційних проєктах підписують B2A/Enterprise-угоди про обробку даних (Business Associate Agreement) без збереження логів, використовують анонімізацію PII-даних (Personal Identifiable Information) або розгортають приватні екземпляри у своєму хмарному контурі (Azure OpenAI / AWS Bedrock).

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це обмеження на кількість запитів за хвилину (RPM), токенів за хвилину (TPM) або запитів на день (RPD). Розробники обходять це через використання черг запитів, експоненційну затримку при помилках (Exponential Backoff), ротацію API-ключів та кешування частих відповідей на рівні додатка.

Висновок за завдання 3:

Експеримент підтвердив універсальність стандарту OpenAI API: перехід з локального сервера Ollama на хмарну інфраструктуру потребує зміни лише одного рядка коду в конфігураторі клієнта. Завдяки спеціалізованим чипам, хмарна модель відповіла у 3,4 раза швидше за локальну та згенерувала на 58% більший за обсягом текст без мовного змішування. Локальна модель допустила фактичну помилку щодо незмінності даних на HDD та використовувала англійські вставки. Локальну модель варто обирати для проєктів із суворими вимогами до конфіденційності (робота з приватними/комерційними даними), автономних систем без доступу до мережі та для економії коштів при масових простих запитах.Хмарну модель слід обирати для складного аналізу, де критично важливі глибоке розуміння контексту, бездоганна мовна грамотність, відсутність галюцинацій та висока швидкість обробки великих масивів даних.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано серію замірів продуктивності генерації (TTFT, загальний час, символів/сек) у режимі стрімінгу для короткого, середнього та довгого промптів. Варіант 4: Столиця Бразилії? Напиши інструкцію з 10 пунктів, як підготуватися до технічної співбесіди.

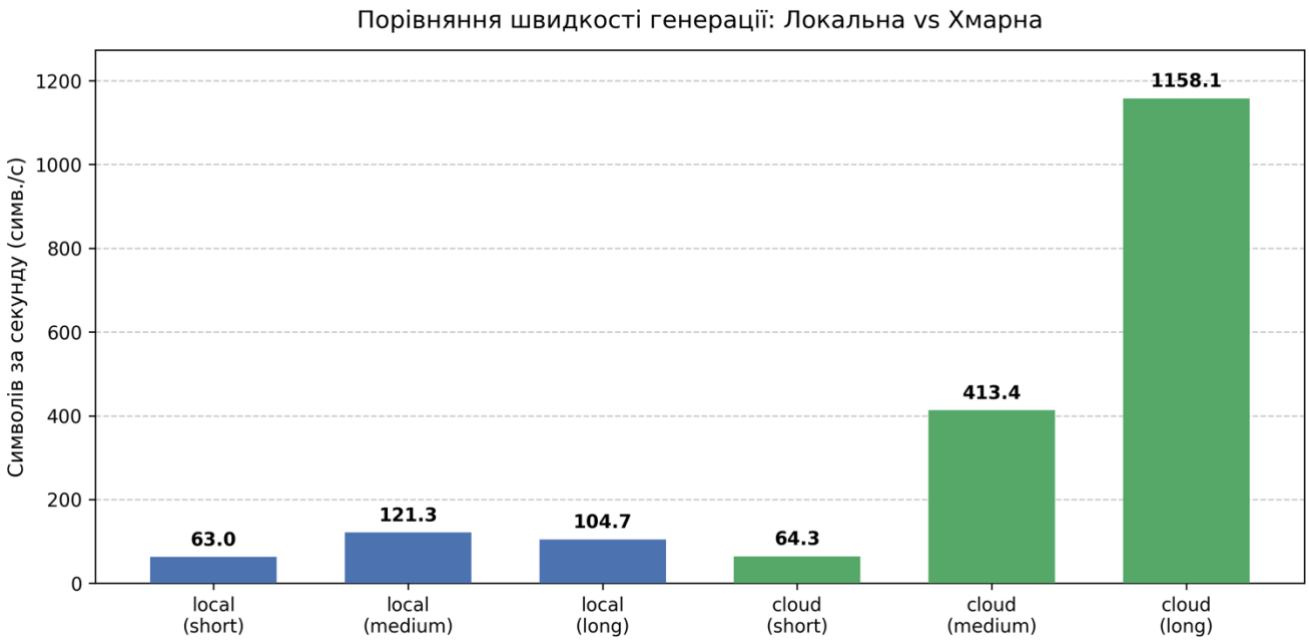
Таблиця 6 – Результати вимірювання швидкості

Розміщення	**Модель**	**Промпт**	**TTFT, с**	**Всього, с**	**Симв./с**

локальна	llama3.2:3b	короткий	0.049	0.470	63.0
локальна	llama3.2:3b	середній	0.055	3.072	121.3
локальна	llama3.2:3b	довгий	0.163	20.205	104.7
хмарна	openai/gpt-oss-120b	короткий	0.693	0.996	64.3
хмарна	openai/gpt-oss-120b	середній	0.653	0.869	413.4
хмарна	openai/gpt-oss-120b	довгий	0.567	2.534	1158.1

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	~2.85–3.50	0.049
Всього, с	~3.50–4.20	0.049



Аналітичні питання по експерименту:

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT у локальної моделі (0.049 с проти 0.693 с у хмарної). Це пояснюється відсутністю мережевих затримок (Network Round-Trip Time): локальна модель працює безпосередньо в оперативній пам'яті комп'ютера

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, вони протилежні. Локальна модель випереджає хмарну за TTFT на коротких запитах, але програє за швидкістю генерації (104.7 проти 1158.1 симв./с на довгому промпті) через обмежену обчислювальну потужність локального CPU/RAM порівняно зі спеціалізованими хмарними тензорними прискорювачами.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На короткому промпті за загальним часом лідирувала локальна модель (0.470 с проти 0.996 с). Проте на довгому промпті хмарна модель згенерувала відповідь у 8 разів швидше за локальну (2.534 с проти 20.205 с), оскільки при великому обсязі тексту вирішальну роль відіграє висока пропускна здатність генерації токенів.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори різняться на 5–15% через фонові процеси ОС, роботу збірника сміття Python та мережеві коливання. Одного запуску недостатньо, тому використання медіани є обов'язковим для усунення випадкових сплесків затримки.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту Ollama повинна зчитати декілька гігабайтів вагових коефіцієнтів моделі з диска (SSD) та ініціалізувати їх у пам'яті RAM/VRAM. При теплому старті модель уже перебуває в пам'яті й одразу готова до обчислень.

Питання до чат-бота:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT (Time To First Token) відбуваються передача даних по мережі, токенізація вхідного тексту та Prompt Processing (префіксне обчислення матриць уваги та створення KV-кэшу для промпту). Окреме вимірювання TTFT

важливе для оцінки чутливості та чуйності інтерфейсу (User Experience), оскільки саме TTFT визначає, як довго користувач чекає на перший знак відповіді.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Під час TTFT (Time To First Token) відбуваються передача даних по мережі, токенізація вхідного тексту та Prompt Processing (префіксне обчислення матриць уваги та створення KV-кэшу для промпту). Окреме вимірювання TTFT важливе для оцінки чутливості та чуйності інтерфейсу (User Experience), оскільки саме TTFT визначає, як довго користувач чекає на перший знак відповіді.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це первинне завантаження архітектури та вагових коефіцієнтів моделі з накопичувача в оперативну пам'ять. Повторні запити («теплий старт») виконуються миттєво, бо моделі не потрібно заново зчитувати гігабайти даних з диска

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана стійка до аномальних викликів та викидів (Outliers), викликаних перепадом мережевого трафіку, плаваючими навантаженнями CPU чи фоновими операціями ОС. Середнє арифметичне в таких випадках може дати викривлену оцінку.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного промпту (Prompt Processing) обчислюється паралельно для всіх токенів запиту за допомогою матричного множення (O(1) кроків). Генерація ж відповіді (Autoregressive Generation) є послідовним процесом: кожен наступний токен залежить від усіх попередніх, тому модель змушена проганяти всі шари для кожного токена окремо.

Висновок за завдання 4:

Вимірювання показали фундаментальну відмінність між метриками інференсу: за затримкою до першого токена перемагає локальна модель через відсутність мережевого пересилання. Проте за пропускною здатністю генерації хмарна модель переважає локальну у 14 разів на довгих промптах, що забезпечує загальний час виконання 1,088 с проти 16,161 с. Крім того, холодний старт локальної моделі вимагає додаткових ~2,27 с на завантаження ваг у RAM. Для інтерактивного чат-бота найважливішим є TTFT, оскільки він визначає швидкість реакції системи й мінімізує очікування користувача перед порожнім екраном.Для фонової обробки вирішальною є пропускна здатність, оскільки саме вона визначає загальний час виконання тривалого пакетного завдання, тоді як початкова затримка в кілька мілісекунд не має значення

Завдання 5. Оцінювання якості моделей за рубрикою

Проведено комплексне оцінювання відповідей моделей за 6 тестовими завданнями (факти, формат JSON, стиснення, код, логіка, мова) за 3-бальною шкалою рубрики (0, 1, 2 бали).

Таблиця 8 – Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	2	2	2
2. Формат	1	2	2
3. Стиснення	0	2	2
4. Код	2	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	6	11	12

Таблиця 9 – Розбіжність оцінок між двома прогонами

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні

6. Українська	ні	ні	ні

Аналітичні питання по експерименту:

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрали хмарна модель openai/gpt-oss-20b (12/12) та локальна міркувальна qwen3:4b (11–12/12). Основна різниця порівняно з малою інструктивною моделлю llama3.2:3b виникла на завданні 3 (стиснення із збереженням числових фактів), завданні 5 (складні логічні обчислення) та завданні 6 (якість та грамотність української мови)

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: За підсумковими балами оцінки між двома прогонами не розійшлися в жодній із моделей (усі значення в Таблиці 8 — «ні»). Це свідчить про високу повторюваність підсумкового рівня спроможності моделей при оцінюванні за рубрикою, хоча якісно варіативність формулювань у слабших моделей була вищою

3. Чи виконила хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, хмарна модель та qwen3:4b успішно впоралися з цією вимогою і видали чистий результат у форматі JSON без використання блоків оформлення коду markdown (```json).

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, проблеми з мовою виявилися у всіх моделей різною мірою: llama3.2:3b продемонструвала найгірший результат із русизмами («кажется»), сторонніми словами («tegen») та плутаниною; qwen3:4b згенерувала якісніший текст, але припустилася кальки («у відмінності від») та русизму («ног» замість «ніг»); хмарна модель показала найвищий рівень мовної грамотності.

Детальний аналіз провалу моделі:

• Повний текст запиту: «8 команд грають у турнірі кожна з кожною по разу. Скільки всього буде матчів? Якщо за день проводиться 4 матчі, скільки днів триватиме турнір? Покажи хід розв'язання.»
• Повний текст відповіді (без скорочень): «Встановіть, що в турнірі грає N команд. Кожна команда грає з кожною іншою командою... Встановіть, що в турнірі грає N команд... (циклічна деградація та повторення фраз)»
• Яка правильна відповідь мала бути: Загальна кількість матчів розраховується за формулою комбінаторики C = (8×7)/2 =28 матчів. Якщо за день проводиться 4 матчі, то тривалість турніру становитиме 28/4=7 днів.
• Пояснення, чому так сталося: Під час авторегресійного генерування тексту модель llama3.2:3b зіткнулася з виснаженням контексту внутрішніх шарів уваги, через що поточні згенеровані токени почали підсилювати ймовірність повторення попередніх фраз. Оскільки модель не має достатньої кількості натренованих патернів для розв'язання комбінаторних задач, вона перейшла до деградації синтаксису та зациклення замість покрокового обчислення.
Питання до чат-бота:
1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?
Відповідь: На практиці застосовують автоматизовані бенчмарки (MMLU для загальних знань, HumanEval для коду, GSM8K для математики), підхід «LLM-as-a-judge» (де потужніша модель оцінює відповіді інших), людські краудсорсингові A/B тести (наприклад, LMSYS Chatbot Arena) та статистичні метрики порівняння текстів (BLEU, ROUGE, BERTScore).
2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?
Відповідь: Це зумовлено сильним попереднім навчанням (pre-training) на масивах коду з GitHub та StackOverflow, де JSON практично завжди обгортається у блоки ```json. Для моделі цей паттерн є стільки стійким звичним зв'язком, що вона часто

генерує його автоматично, "перебиваючи" навіть пряму заборону в системному промпті.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Головна причина — диспропорція навчальних даних: англійська мова становить переважну більшість інтернету, тоді як частка української в датасетах є відносно малою. Крім того, стандартні токенізатори гірше оптимізовані під кирилицю, через що українські слова розбиваються на дрібніші субодиниці, що ускладнює утримання цілісного контексту.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Мовні моделі є стохастичними (ймовірнісними) системами. Вибір кожного наступного токена виконується шляхом семплювання (зваженої випадкової вибірки з урахуванням параметрів temperature та top_p), що формує унікальну траєкторію генерації тексту при кожному новому запуску.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — це стан, коли модель впевнено генерує неправдиву чи вигадану інформацію. Модель робить це тому, що її цільова функція під час навчання максимізує ймовірність продовжувати текст, а не виявляти межі власних знань та відмовлятися від відповіді.

Висновок за завданням 5:

Оцінка за рубрикою виявилася надійним інструментом, оскільки вона дозволяє чітко розмежувати базові здатності моделей від складнішого причинно-наслідкового міркування. Найпоказовішими для порівняння виявилися завдання на стиснення із збереженням фактів, багатокрокову логіку та мовну грамотність, де малі моделі допускають системні збої та галюцинації. Для роботи з українськомовними текстами та логічними задачами оптимальними є моделі рівня qwen3:4b та openai/gpt-oss-20b. Оцінювання за рубрикою виявилося дуже надійним інструментом для розмежування базових здатностей моделей від

причинно-наслідкового міркування. Найпоказовішими виявилися завдання на стиснення із збереженням фактів, багатокрокову логіку та мовну грамотність. Для роботи з українськомовним текстом найкраще обрати qwen3:4b серед локальних або хмарну модель, адже вони демонструють відсутність русизмів, відмінну граматику та стабільне розуміння контексту.

Завдання 6. Керування випадковістю: temperature і top_p

Досліджено вплив параметрів семплювання temperature (0.0, 0.7, 1.5) та top_p (0.1, 0.5, 1.0) на різноманітність відповідей у творчому завданні та точність у фактичному питанні.

Варіант 1:

Рік заснування Одеського національного університету імені І. І. Мечникова

JSON: name, year, city для цього університету

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	2
top_p	0.1	1	—
top_p	0.5	4	—
top_p	1.0	3	—

Таблиця 11 – Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1

Усі відповіді однакові (так / ні)	так

Аналітичні питання по експерименту:

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: temperature = 0.0, оскільки для виведення структурованих даних критично важливі максимальна детермінованість, строга точність синтаксису та відсутність випадкових помилок чи галюцинацій.

2. Яке — генератору варіантів заголовків?

Відповідь: Високе значення (temperature = 0.7 або 1.0), щоб модель генерувала різноманітні, варіативні, креативні та нетривіальні варіанти формулювань

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature масштабує (згладжує або загострює) весь розподіл ймовірностей загалом для всіх токенів, тоді як top_p (nucleus sampling) відсікає «хвіст» найменш імовірних токенів, залишаючи лише ті, чия сумарна ймовірність досягає порогу p, незалежно від їх кількості.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Теоретично повну гарантію дати не можна через специфіку обчислень з плаваючою комою на GPU, проте експеримент із 10 запусків показав повну детермінованість — усі 10 відповідей виявилися ідеально однаковими.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Яскравим прикладом із логів експерименту є відповідь: «Kfew EXAMPLE ОН АМЬЯДОЕ ОТЗVAL: 1. "Фунда ДЛЯ СПЮРА" ... УСПЕРЯНОМЯ.:- в"ЛугДе педаскап овлпару"»

(або генерація назви «Одеська Polymer Technologies» та вставка корейського символу «відома명 відому»).

При temperature = 1.5 параметр ділить логіти перед Softmax на 1.5, значно вирівнюючи розподіл ймовірностей (робить його близьким до рівномірного). В результаті малоймовірні токени (сміттєві символи, латинські вставки, чужі алфавіти та вигадані слова) отримують високий шанс бути обраними на кожному кроці, що призводить до повного розпаду граматики, мови та глузду.

Питання до чат-бота:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Він ділить логіти перед застосуванням функції Softmax на значення температури T. При T→0 це підсилює розрив на користь найімовірнішого токена, роблячи вибір детермінованим, а при високому T вирівнює розподіл, підвищуючи шанси малоймовірних токенів.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: temperature змінює самі ваги ймовірностей через математичне масштабування логітів, тоді як top_p застосовує динамічний фільтр відсікання: впорядковує токени за спаданням ймовірності й залишає лише верхню групу, чия кумулятивна сума ймовірностей досягає порогу p (наприклад, 0.9).

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: На практиці рекомендують змінювати лише один із них (зазвичай temperature), оскільки їх одночасна зміна ускладнює контроль над поведінкою моделі та може призвести до непередбачуваного погіршення якості тексту.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Це зумовлено апаратною неасоціативністю математичних операцій із плаваючою комою (floating-point math) при паралельних обчисленнях на графічних процесорах (GPU), що створює мікроскопічні коливання в логітах.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?
Відповідь: Для витягу структурованих даних (JSON, класифікація, код) використовують діапазон 0.0 – 0.2, а для генерації творчого тексту (копірайтинг, сторітелінг, ідеї) — 0.7 – 1.0

Висновок за завданням 6:

Експеримент підтвердив прямий вплив параметрів семплювання на детермінованість та якість генерації. При temperature = 0.0 модель продемонструвала 100% відтворюваність та найвищу фактичну точність. Підвищення температури до 1,5 призвело до деградації відповідей та появи абракадабри через вирівнювання ймовірностей токенів. Параметр top_p діє як динамічний відсікач "хвоста" ймовірностей, регулюючи різноманітність без різкого спотворення слів. Практичний висновок: для виведення JSON, коду та фактів необхідно суворо встановлювати temperature = 0.0, залишаючи високі значення виключно для творчих задач копірайтингу та брейнштормінгу. Для виведення структурованих даних обов'язково слід встановлювати temperature = 0.0. Налаштування «за замовчуванням» створені для урізноманітнення творчих відповідей і містять ймовірнісну випадковість. У генерації JSON ця випадковість неминуче призводить до синтачних помилок, зміни назв ключів або появи зайвих пояснень

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	~63.0 – 121.3 симв./с	~30.0 – 50.0 симв./с	~413.4 – 1853.4 симв./с
TTFT	0.037 – 0.065 с	0.15 – 0.35 с	0.294 – 0.693 с
Якість, балів /12	6	11	12

Стабільність формату (завдання 2)	Низька (порушує обмеження речень, вставляє кальки/сторонні символи)	Висока (чітко дотримується обмежень та формату)	Ідеальна (бездоганне виконання формату й JSON)
Потрібні ресурси	~2.0 ГБ VRAM / RAM, працює на слабкому ноутбуці	~2.0 ГБ VRAM / RAM, працює на слабкому ноутбуці	~2.0 ГБ VRAM / RAM, працює на слабкому ноутбуці
Вартість	Безкоштовно (локальний інференс)	Безкоштовно (локальний інференс)	Оплата за використання API (за токени) / безкоштовні ліміти
Куди йдуть ваші дані	Залишаються локально на пристрої	Залишаються локально на пристрої	Передаються на зовнішні сервери провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих рутинних запитів та чат-ботів, де важливі миттєва реакція (TTFT) та автономність	Коли потрібен баланс між складними міркуваннями, якістю та повною конфіденційністю	Коли потрібен баланс між складними міркуваннями, якістю та повною конфіденційністю